

STRINGS

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Introdução
- Leitura de Caracteres
- Definição de Strings
- Leitura de Strings
- Exibição de Strings
- Funções úteis para Strings
- Dúvidas



INTRODUÇÃO

- As variáveis do tipo **char** (1 byte) são especificadas por apenas um único caractere alfanumérico delimitado por aspas simples.
- Exemplo: `char letra = 'a';`
- Neste caso **letra** contém o caractere **a** que representa o código 97 em ASCII.
- Portanto:

```
printf("%c",letra); //saída na tela: a  
printf("%i",letra); //saída na tela: 97
```

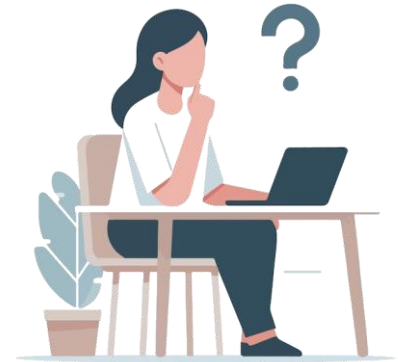
LEITURA DE CARACTERES

- O tipo **char** representa caracteres, que internamente são armazenados como códigos inteiros da tabela ASCII.
- Dígitos **0–9** : códigos 48 a 57
- Letras maiúsculas **A–Z** : códigos 65 a 90
- Letras minúsculas **a–z** : códigos 97 a 122
- Exemplo:

```
for (i=32; i<128; i++) {
```

```
    printf("%d = %c\n", i, i);  
}
```

Imprime todos os caracteres de 32 até 127 (caracteres imprimíveis da ASCII)



LEITURA DE CARACTERES

- Quando um caractere é digitado e na sequência pressiona-se a tecla enter, esta também gera um caractere (`\n`) que também é armazenado no buffer de entrada, ou seja, o C considera o *enter* como uma segunda letra.
- A solução para isso é usar no primeiro argumento da função `scanf()` um espaço entre a aspa " e o símbolo `%c`.
- O simples espaço é um comando para o C desconsiderar o enter, tab ou espaço em branco.
- **Exemplo:** `char letra;`

```
scanf(" %c", &letra);
```



LEITURA DE CARACTERES

- Quando utiliza-se as funções `getchar()`, `getc()` e `fgetc()` para ler caracteres do teclado, a tecla *enter* (`\n`) também é armazenada no buffer de entrada.
- Isso pode causar problemas em leituras seguintes, pois o `\n` é interpretado como um caractere válido.
- Para limpar o buffer no Windows utiliza-se o `fflush(stdin)` e Linux(`glibc`), `__fpurge(stdin)`
- Uma solução mais portátil para essa questão é mostrada abaixo.
- Exemplo: `int ch;`

```
while ((ch = getchar()) != '\n' && ch != EOF);
```



lê e descarta tudo até o `\n`

LEITURA DE CARACTERES

- Comparando as funções `getchar` e `getc`:
 - `getchar()`: lê um caractere do teclado (`stdin`).
 - `getc (FILE *f)`: lê um caractere de qualquer arquivo/fluxo.
- No código, o `getchar()` foi utilizado por ser direto e claro, mas internamente é equivalente a `getc(stdin)`.
- A solução alternativa é mostrada abaixo.
- Exemplo: `int ch;`

```
while ((ch = getc(stdin)) != '\n' && ch != EOF);
```



lê e descarta tudo até o `\n`

LEITURA DE CARACTERES

- O caractere pode ser tratado como **número**.
- Operações matemáticas com char resultam em novos caracteres.
- Exemplo: `char c='a';`
- `printf("%c - %c", c, c+1); // a - b`



LEITURA DE CARACTERES

- A biblioteca `<ctype.h>` contém funções para testar e manipular caracteres individuais, facilitando operações como verificação de letras, números, maiúsculas dentre outras.
 - **isalpha()** - verifica se o caractere é uma letra (A-Z ou a-z)
 - **isdigit()** - verifica se o caractere é um dígito numérico (0-9)
 - **isupper()** - verifica se o caractere é maiúsculo (A-Z)
 - **islower()** - verifica se o caractere é minúsculo (a-z)
 - **toupper()** - converte um caractere em maiúsculo
 - **tolower()** - converte um caractere em minúsculo



DEFINIÇÃO DE STRINGS



- Como variáveis do tipo char armazenam apenas um único caractere, quando é necessário guardar dois ou mais caracteres deve-se utilizar uma **string**.
- Uma **string** é uma sequência de caracteres, ou seja, um vetor de char, delimitado por aspas duplas (" ").
- Sempre que o compilador encontra aspas duplas, ele reconhece uma string constante e adiciona automaticamente um caractere nulo (`\0`) ao final.
- O comprimento de uma string é determinado contando-se os caracteres até o `\0`, sem incluí-lo. Esse caractere final indica o fim da sequência.

DEFINIÇÃO DE STRINGS

- Para **armazenar** uma string deve-se, portanto, declarar um vetor do tipo char com a quantidade de elementos desejada e adicionar um.

a	m	i	g	o	\0
texto[0]	texto[1]	texto[2]	texto[3]	texto[4]	texto[5]

- **Sintaxe:** `char nome_variavel[numero_elementos+1];`
- Para inicializar uma string seriam possíveis algumas alternativas:
- **Exemplo:** `char texto[6]={'a','m','i','g','o'};`
`char texto[6]="amigo";`
`char texto[ ]="amigo";`

LEITURA DE STRINGS

- Para **ler** uma string pode-se utilizar: `scanf`, `gets` ou `fgets`, sendo todas da biblioteca `<stdio.h>`.
- A função **`scanf()`** é limitada para leitura de string, pois ela utiliza qualquer espaço em branco (espaço, caractere de tabulação ou nova linha) para finalizar a entrada.
- **Sintaxe:** `scanf("%s", &nome_string);`
- **Exemplo:** `char string[100];`

```
printf("Digite o seu nome: ");
```

```
scanf("%s", string); //nome é seu endereço inicial
```

```
printf("\n\nOi %s", string);
```

LEITURA DE STRINGS

- A função **gets()** lê strings do teclado e é conveniente para leitura de string, pois ela lê os caracteres até encontrar o caractere de nova linha (`\n`), que é gerado quando a tecla *enter* é pressionada.
- Logo caracteres em branco são aceitos como parte da string.
- **Sintaxe:** `gets(nome_string);`
- **Exemplo:**

```
char string[100];  
  
printf("Digite o seu nome: ");  
  
gets(string);  
  
printf("\n\nOi %s", string);
```

LEITURA DE STRINGS

- A função **fgets()** lê strings do teclado (padrão) ou de um arquivo.
- A função fgets() é conveniente para leitura de string, pois ela lê os caracteres até encontrar o caractere de nova linha (\n), que é gerado quando a tecla *enter* é pressionada ou até que o limite especificado (num-1) seja atingido, evitando que se escreva além do permitido.

- **Sintaxe:** fgets(nome_string,num,entrada);

- **Exemplo:** char string[9];

```
printf ("Digite o seu nome: ");
```

```
fgets (string,9,stdin);
```

```
printf ("\n\nOi %s",string);
```

num: lê num-1 caracteres

entrada: stdin ou arquivo

EXIBIÇÃO DE STRINGS

- Para **mostrar** uma string pode-se utilizar: `printf`, `puts` ou `fputs` , sendo todas da biblioteca `<stdio.h>`.
- A função **`printf()`** permite exibir mais de uma string, ou ainda mostrar textos ou variáveis numéricas na mesma linha.
- **Sintaxe:** `printf("%s", nome_string);`
- **Exemplo:**

```
char produto[100] = "Notebook";  
char loja[100] = "Tech Store";  
float preco = 3499.90;  
  
printf("O produto %s está disponível na %s por R$  
%.2f.\n", produto, loja, preco);
```

EXIBIÇÃO DE STRINGS

- A função **puts()** permite exibir apenas uma string por vez na tela e pula de linha automaticamente após imprimir uma string.

- **Sintaxe:** `puts(nome_string);`

- **Exemplo:**

```
char produto[100] = "Notebook";  
char loja[100] = "Tech Store";  
float preco = 3499.90;  
puts("Produto:");  
puts(produto);  
puts("Loja:");  
puts(loja);  
printf("Preço: R$ %.2f\n", preco);
```

EXIBIÇÃO DE STRINGS

- A função **fputs()** é usado quando é necessário mais controle, pois não pula linha automaticamente e permite escrita em arquivos.

- **Sintaxe:** `fputs(nome_string, stdout);`

- **Exemplo:**

```
char produto[100] = "Notebook";  
char loja[100] = "Tech Store";  
float preco = 3499.90;  
fputs("Produto:", stdout);  
fputs(produto, stdout);  
fputs("\nLoja:", stdout);  
fputs(loja, stdout);  
printf("\nPreço: R$ %.2f", preco);
```

FUNÇÕES ÚTEIS PARA STRINGS

- Existem operações que não se pode fazer diretamente com strings, pois string não é um tipo nativo na linguagem C.
- A biblioteca `<string.h>` contém uma série de funções para manipular strings.
- As principais são:
 - `strlen()`
 - `strcpy()` e `strncpy()`
 - `strcmp()` e `strncmp()`
 - `strcat()` e `strncat()`



FUNÇÕES ÚTEIS PARA STRINGS

- Para se encontrar a quantidade de caracteres de uma string pode-se utilizar um algoritmo com estrutura de repetição.

- Exemplo:

```
int conta_caracteres(char str[]) {  
    int cont=0;  
    while(str[cont]!='\0')  
        cont++;  
    return cont;  
}
```

Caractere `'\0'` finaliza a string



FUNÇÕES ÚTEIS PARA STRINGS

- A função **strlen()** retorna o tamanho da string. Ela identifica o tamanho da sua string assim que encontra o caractere zero (`\0`), ou seja, não conta o caractere `\0`.
- **Sintaxe:** `var_int = strlen (string);`
- **Exemplo:** `int s;`

```
char nome[ ] = "Mariana da Silva";
```

```
s = strlen (nome); // s conterà o valor 16
```

FUNÇÕES ÚTEIS PARA STRINGS

- Para realizar a atribuição entre duas strings, não é possível realizar uma atribuição direta.

- **Exemplo 1:** `char str1[10], char str2[10]="Notebooks";`

`str1=str2;`



- **Exemplo 2:** `char str1[10], str2[10]="Notebooks";`

`int i;`

`for(i=0;i<10;i++){`

`str1[i]=str2[i];`

`}`



FUNÇÕES ÚTEIS PARA STRINGS

- A função **strcpy()** é utilizada sempre que se deseja copiar uma string em outra. Dessa forma, cada caractere de uma string é copiado para a outra string.

- **Sintaxe:** `strcpy(string_destino, string_origem);`

- **Exemplo:** `char str1[100],str2[100];`

```
printf ("Entre com uma string: ");
```

```
gets (str1);
```

```
strcpy (str2,str1); // Copia str1 em str2
```

```
printf ("\n\n%s",str2);
```



FUNÇÕES ÚTEIS PARA STRINGS

- A função **strncpy** realiza a cópia do conteúdo de uma variável a outra, porém, deve ser especificado o tamanho a ser copiado, ou seja, trabalha com um número fixo de caracteres.
- **Sintaxe:** `strncpy(string_destino, string_origem, tamanho);`

FUNÇÕES ÚTEIS PARA STRINGS

- Para realizar a comparação entre duas strings, não é possível realizar uma comparação direta
- **Exemplo:** `char str1[10], char str2[10]="Notebooks";`
`if(str1==str2)` 
`printf("Strings iguais");`

FUNÇÕES ÚTEIS PARA STRINGS

- A função **strcmp()** é utilizada quando se deseja comparar strings. Nesse caso, o valor de retorno pode ser:
 - zero (0): caso as duas strings sejam iguais
 - negativo (-1): caso a string1 seja menor que a string2
 - positivo (1): caso a string1 seja maior que a string2
- **Sintaxe:** `var_int = strcmp(string1, string2);`
- **Exemplo:**

```
printf("%i\n", strcmp("A", "A")); //Exibe inteiro 0
printf("%i\n", strcmp("A", "B")); //Exibe inteiro -1
printf("%i\n", strcmp("B", "A")); //Exibe inteiro 1
```



FUNÇÕES ÚTEIS PARA STRINGS

- A função **strncmp()** também faz a comparação do conteúdo de duas strings, porém, deve ser especificado o tamanho a ser comparado.
- **Sintaxe:** `var_int = strncmp(string1, string2, tamanho);`
- **Exemplo:**

```
if (strcmp(s1,s2)==0)
    printf("Todos os caracteres sao iguais\n");
if (strncmp(s1,s2,5)==0)
    printf("Os 5 primeiros caracteres sao iguais\n");
```

FUNÇÕES ÚTEIS PARA STRINGS

- A função **strcat()** concatena duas strings, adicionando o conteúdo da segunda ao final da primeira, além do caractere NULL (`\0`).
- A primeira string deve ter espaço suficiente para conter a segunda, para que não ocorra um estouro de buffer.
- **Sintaxe:** `strcat(string_destino, string_origem);`
- **Exemplo:**

```
char titulo[50] = "Strings-";  
char subtitulo[ ] = " Funcoes de Manipulacao";  
strcat (titulo, subtitulo);  
printf("\nAula: %s", titulo);
```



FUNÇÕES ÚTEIS PARA STRINGS

- A função **strncat()** realiza a concatenação do conteúdo de uma variável a outra, porém, deve ser especificado o tamanho a ser concatenado.
- **Sintaxe:** `strncat(string_destino, string_origem, tamanho);`

EXERCÍCIOS

- **Ex1** - Inicialize uma string e mostre a quantidade de letras maiúsculas, minúsculas e dígitos contidos na string.
- **Ex2** - Solicite ao usuário que digite uma senha composta por 6 caracteres e na sequência pergunte novamente a senha. Confira se as duas sequências de caracteres são iguais e em caso afirmativo, exiba uma mensagem dizendo que a senha é válida. Caso os três primeiros caracteres sejam iguais, informe que houve algum erro na digitação, caso contrário, a senha será inválida.



EXERCÍCIO 1

```
#include <stdio.h>
#include <string.h>
int main() {
    char texto[ ] = "Strings 2025!";
    int i, tam, maiusculas = 0, minusculas = 0, digitos = 0;
    tam=strlen(texto);
    for (i = 0; i<tam; i++) {
        if (isupper(texto[i]))
            maiusculas++;
        else if (islower(texto[i]))
            minusculas++;
        else if (isdigit(texto[i]))
            digitos++;
    }
    printf("Maiusculas: %d\nMinusculas: %d\n", maiusculas, minusculas);
    printf("Digitos   : %d\n", digitos);
    return 0;
}
```



EXERCÍCIO 2

```
#include <stdio.h>
#include <string.h>
int main() {
    char senha1[7], senha2[7];

    printf("Digite a sua senha: ");
    fgets(senha1, 7, stdin);
    printf("Confirme a sua senha: ");
    fgets(senha2, 7, stdin);

    if (strcmp(senha1, senha2) == 0)
        printf("Senha valida!\n");
    else if (strncmp(senha1, senha2, 3) == 0)
        printf("Apenas 3 primeiros caracteres corretos.\n");
    else
        printf("Senha invalida!\n");
    return 0;
}
```



DÚVIDAS

- Questionamentos?

